Early Detection of Silent Training Failures in Neural Networks

Sohan Bhat

October 2026

Abstract

During training, neural networks can develop defects, yet nothing fails visibly since the run completes normally and simply produces a degraded result; these are called "silent failures." Since training runs consume significant compute, every silent failure that goes undetected wastes it. This work proposes LHD (Layer Health Diagnostics), a monitor that reads the raw numbers in the weights and biases, catches the failure within a few epochs, identifies the broken layer and its likely cause, and halts the run before compute is wasted. LHD logs four signals for every layer at the end of each epoch: weight change, gradient norm, weight norm, and the zero-activation fraction. Thresholds are set just outside the range each signal stays within during healthy runs, and broken runs are never used to set them. A fault is then introduced into one chosen layer, and for every broken run the epoch at which LHD fires is compared with the epoch at which the accuracy curve alone would raise suspicion. LHD is also run on fresh, healthy networks, where every incorrect flag is counted and reported as a false alarm rate. The full procedure is repeated on a CNN, a fully connected network, and a small transformer.

1 Introduction

During training, neural networks can develop defects, yet nothing fails visibly since the run completes normally and simply produces a degraded result; thus, these are called "silent failures." Because of this, identifying whether the degraded result was caused by a difficult problem or a silent failure becomes difficult; therefore, it is common to misread a failure as normal. I initially encountered this as I was training my small-scale GTSRB (German Traffic Sign Recognition Benchmark) model (Stallkamp et al., 2012) from scratch, where my model plateaued at 76%, wasting weeks tuning it, until I found one convolutional layer had been accidentally excluded from the optimizer and was never properly trained. Fixing this minute detail raised the accuracy to 87%. This is not an isolated case: errors in deep learning implementations rarely cause an exception and instead reveal themselves only through degraded predictive performance (Chwilczyński et al., 2024).

Since training runs consume significant compute, every silent failure that goes undetected wastes it. The evidence existed in the form of raw numbers in the weights and biases, but perceiving that is challenging since nothing in standard practice surfaces them. To address this, this work proposes LHD (Layer Health Diagnostics), a monitor that reads these numbers, catches the failure within a few epochs, identifies the broken layer and its likely cause, and halts the run before compute is wasted.

2 Related Work

Karpathy's Recipe. In 2019, Andrej Karpathy wrote a widely cited guide for the training of neural networks where he discusses how training fails silently (Karpathy, 2019). He provides manual habits to catch these silent failures, and the Stanford CS231n course notes add checking that the update-to-weight ratio is

near 0.1% (CS231n, n.d.). However, his guide simply provides a checklist a human needs to follow by hand with no mention of any automated solution.

wandb.ai. Weights & Biases (wandb) is the industry standard for experiment tracking through a dashboard where one line of code in a project can log live gradient and weight histograms during training (Biewald, 2020). Still, it is limited by only displaying the numbers and nothing more, since a human still needs to watch, understand what healthy numbers look like, and catch the anomaly.

wandb-callbacks. Wandb-callbacks is a small community add-on package for wandb that includes a dead-ReLU warning callback (Groeger, n.d.). However, it covers only one failure mode with a single signal, provides no diagnosis or halting, and is an unmaintained hobby package rather than a supported tool.

ART. Actually Robust Training is a Python library inspired by Karpathy's recipe that adds validation checks to training pipelines (Chwilczyński et al., 2024). While this is a step up, it only imposes process steps at development time rather than live fault detection from internal signals during training.

To conclude, while tools are developed that aid in the process of training neural networks, no existing tool automates fault detection end to end. Furthermore, no systematic measurement exists of how many epochs earlier internal signals catch a fault than the accuracy curve or how often they raise false alarms.

3 Methodology

This work hypothesizes that per-layer internal signals (weight change, gradient norm, weight norm, and fraction of zero outputs) will expose a broken layer within the first few epochs, before the accuracy curve shows anything wrong, without flagging healthy runs.

3.1 Proposed Approach

LHD logs these four signals for every layer at the end of each epoch: weight change, gradient norm, weight norm, and the zero-activation fraction. The weight change captures how much the layer's weights moved since the previous epoch, so a frozen layer reads exactly zero. Gradient norm measures the overall size of the learning signal; a vanishing gradient reads near zero. Similarly, weight norm reflects the overall size of the layer's weights, which reveals runaway growth before divergence. The zero-activation fraction represents how much of a layer's outputs are exactly zero, so dead units read near one. Furthermore, there is minimal overhead doing these operations since the numbers that represent these different signals are already in memory.

A signal value is only meaningful relative to what is normal for that layer. Normal is measured by training the network several times, with no defects introduced, from different random starting points and recording the range each signal stays within. The threshold is set just outside that range, and a layer is flagged only after numerous consecutive epochs beyond it. Broken runs are never used to set thresholds because a detector tuned on a known fault only catches that fault.

As previously mentioned, the wandb-callbacks package only detects a dead ReLU and issues a warning. However, detection alone names a layer but does not suggest why the layer is broken and how to fix it. Naming a cause involves reading the pattern across all four signals. For instance, a layer with a healthy gradient but zero weight change is receiving instruction but not applying it to the weights. On the contrary, when a layer's neurons begin to die, its zero-activation fraction climbs toward one while its gradient and weight change decay toward zero over the following epochs.

Once a fault is confirmed, LHD stops the run and reports the flagged layer, the signal pattern observed, and the likely cause with a suggested check. As a result, the failure is caught within the first few epochs instead of consuming the entire run, so nearly all the compute that would have been wasted is saved.

3.2 Experimental Design

The CNN is first trained several times with no defects introduced, each run starting from a different random starting point. The healthy range of each signal is recorded for every layer, and thresholds are set just outside those ranges.

A fault is then introduced into one chosen layer, one fault per run. The frozen layer is tested first, followed by dead neurons. Because each break is planted deliberately, the correct answer for both the layer and the cause is known in advance.

For every broken run, two epochs are recorded. The first is the epoch at which LHD fires, and the second is the epoch at which the accuracy curve alone would raise suspicion. The gap between the two is the primary result, along with whether LHD named the correct layer and cause.

LHD is then run on fresh, healthy networks that played no part in setting the thresholds. Every incorrect flag is counted and reported as a false alarm rate.

Finally, the full procedure is repeated on a fully connected network and a small transformer. Only the baselines are recalibrated for each architecture, while the detection rules remain untouched.

References

Biewald, L. (2020). Experiment tracking with Weights and Biases. Software available from wandb.com. https://www.wandb.com/

Chwilczyński, S., Trębacz, K., Cyganik, K., Małecki, M., and Brzezinski, D. (2024). ART: Actually Robust Training. In *Machine Learning and Knowledge Discovery in Databases. Research Track and Demo Track (ECML PKDD 2024)*, Lecture Notes in Computer Science, vol. 14948, pp. 374–378. Springer, Cham. https://doi.org/10.1007/978-3-031-70371-3_23. Preprint: arXiv:2408.16285.

CS231n (n.d.). Neural networks part 3: Learning and evaluation. Course notes for CS231n, Stanford University. https://cs231n.github.io/neural-networks-3/

Groeger, F. (n.d.). wandb-callbacks: Additional callbacks for Weights & Biases. GitHub repository. https://github.com/FabianGroeger96/wandb-callbacks

Karpathy, A. (2019). A recipe for training neural networks. Blog post, April 25, 2019. https://karpathy.github.io/2019/04/25/recipe/

Stallkamp, J., Schlipsing, M., Salmen, J., and Igel, C. (2012). Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition. *Neural Networks*, 32:323–332. https://doi.org/10.1016/j.neunet.2012.02.016